POLITECNICO
MILANO 1863

Autonomous Navigation and Path Planning for a TurtleBot3 Robot

Author:
Matteo FILIPPI

Teacher:
Stefano ARRIGONI

Academic year 2024/2025

Contents

1 Introduction **1**

2 Mapping the environment **2**
2.1 Description of the environment . . . 2
2.2 Mapping process . . . 3
2.3 Challenges and limitations . . . 4

3 A* algorithm implementation **6**
3.1 Map representation in MATLAB . . . 6
3.2 Optimizations in the A* algorithm . . . 8
3.2.1 Elimination of the full cost matrix . . . 8
3.2.2 Introduction of sparse matrices . . . 9
3.2.3 Sparse representation of the map . . . 9
3.3 Algorithm considerations . . . 9

4 Pathfinding results and algorithm results visualization **11**
4.1 Path 1 calculation . . . 12
4.2 Path 2 calculation . . . 13
4.3 Path 3 calculation . . . 14
4.4 Path 4 calculation . . . 15
4.5 Path 5 calculation . . . 16
4.6 Path 6 calculation . . . 17

5 Feedback control for trajectory tracking **18**
5.1 Control system architecture . . . 18
5.1.1 Initial acceleration control . . . 20
5.1.2 Final deceleration control . . . 21
5.2 Control system conclusion . . . 21

6 Trajectory tracking results **22**
6.1 Time estimation for Gazebo simulation . . . 22
6.2 Path tracking results . . . 23
6.2.1 Path 1 execution . . . 24
6.2.2 Path 2 execution . . . 26
6.2.3 Path 3 execution . . . 28
6.2.4 Path 4 execution . . . 30
6.2.5 Path 5 execution . . . 32
6.2.6 Path 6 execution . . . 34

7 Discussion and conclusions **36**

Chapter 1

Introduction

In robotics, one of the key challenges for autonomous navigation is the ability to effectively map the environment and plan paths that allow a robot to reach its goals efficiently and safely.
In this project, the goal is to map an indoor environment using a TurtleBot3 robot within a simulated environment and implement an optimized A^* path planning algorithm capable of handling large maps (at least 300x300 pixels).
The implementation is carried out using ROS (Robot Operating System) as the middleware for communication, Gazebo as the simulation platform, and MATLAB for additional computations, analysis, and as a control system.
Mapping the environment is performed using `gmapping`, a SLAM (Simultaneous Localization and Mapping) algorithm that allows the robot to construct a 2D occupancy grid map while being manually controlled through `teleop_node`. Once the map is generated, an optimized version of the A* algorithm is developed to efficiently compute paths between different start and goal points.
The algorithm must be able to handle significantly larger maps than in previous assignments, improving computational efficiency and scalability. Thus, given the increased complexity of the problem, optimizations must be introduced to ensure that the algorithm can efficiently process large-scale maps without excessive computational overhead.
Additionally, as a bonus objective, feedback control mechanisms will be explored to enable the TurtleBot3 robot to autonomously follow the computed paths within the simulated environment.
This requires integration between the planned path and motion control strategies to ensure accurate trajectory tracking.
In summary, the main tasks to be accomplished in this project are:

- Map the environment using `teleop_node` and `gmapping`.
- Implement an optimized A* algorithm for large maps (300x300 pixels or more).
- Apply the algorithm to at least four different start-end point pairs.
- Bonus request: implement a feedback control to make the robot follow the computed path.

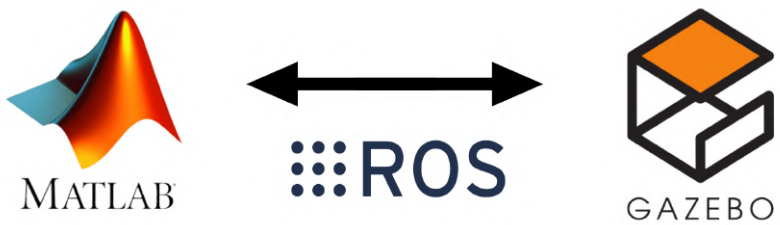

Figure 1.1: Software interconnections via ROS

Chapter 2

Mapping the environment

Accurate environmental mapping is a crucial step in mobile robotics, as it provides the necessary spatial information for autonomous navigation and path planning.
In this project, mapping is performed using the `gmapping` package in ROS, which employs a SLAM (Simultaneous Localization and Mapping) algorithm. The robot explores the environment while continuously constructing a 2D occupancy grid map based on LIDAR sensor data.
This process is essential to ensure that the generated map captures all navigable spaces and potential obstacles, enabling effective path planning in later stages.

2.1 Description of the environment

The environment used for this project is the TurtleBot3 House simulation, which is part of the TurtleBot3 simulation package within ROS. The house consists of multiple rooms, each with various pieces of furniture and obstacles that the robot must properly detect and map. An image of the simulated environment is reported to provide a visual reference of the layout and objects present.

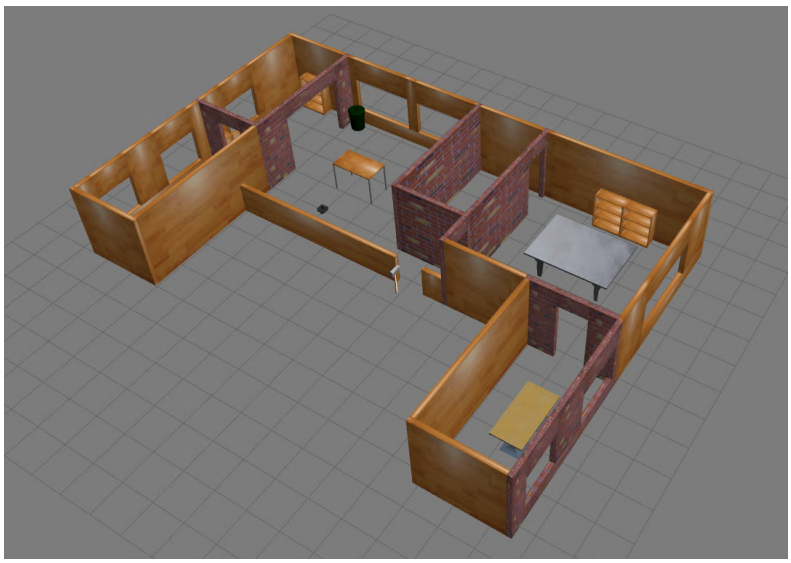

Figure 2.1: TurtleBot3 House environment

2.2 Mapping process

To construct an accurate map, the TurtleBot3 robot was manually controlled using `teleop_node` and moved systematically throughout the house.
The mapping process relied on LIDAR-based SLAM, where the robot scanned its surroundings using a 2D LIDAR sensor and incrementally built an occupancy grid map.
The LIDAR sensor emits laser beams that measure distances to surrounding objects.
These measurements are then used to classify areas as free space, occupied space (obstacles), or unknown space in the occupancy grid map. The result is a detailed representation of the environment, which serves as the foundation for path planning.
During the mapping process, the robot was moved slowly and methodically to ensure that all areas were properly scanned. Each room and corridor was carefully explored, covering the entire accessible space to create a complete and accurate representation of the environment.
Special attention was given to maintaining a steady pace and avoiding sudden rotations or rapid movements, as these could introduce errors in the SLAM algorithm and compromise the quality of the generated map.
However the initial mapping attempts were not satisfactory, as the generated maps lacked precision and reliability. To improve the results, it was decided to slightly modify the `gmapping` parameters. By adjusting the update thresholds and increasing the number of particles, the system achieved better localization accuracy and a higher-quality map.

- `linearUpdate` was reduced to 0.05 to allow more frequent updates in response to small movements;
- `angularUpdate` was decreased to 0.01 to improve rotational accuracy;
- `minimumScore` was increased to 200 to ensure only high-confidence scans were accepted;
- `particles` was raised to 200 to enhance the robustness of the particle filter.

The table below compares the default parameters with the modified values:

Parameter	**Default value**	**Modified value**
`linearUpdate`	0.2	0.05
`angularUpdate`	0.5	0.01
`minimumScore`	50	200
`particles`	30	200

Table 2.1: Comparison of default and modified `gmapping` parameters

After the robot has successfully explored all the areas of interest, the generated map is saved using the terminal command:

```
rosrun map_server map_saver -f test_map
```

This command creates a `.pgm` file with a resolution of 5 cm per pixel, representing the mapped environment as a grayscale occupancy grid.
Particular attention was also given to mapping the exterior of the house to ensure that the pathfinding algorithm could handle trajectories that include outdoor areas.
The final result of the mapping process is shown in the following figure.

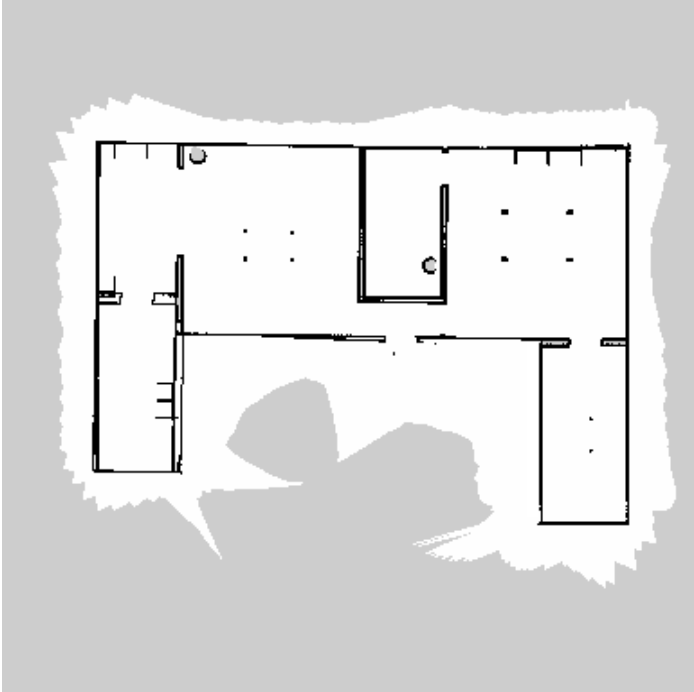

Figure 2.2: House map

2.3 Challenges and limitations

One of the key challenges in mapping the environment is the limitation related to the LIDAR sensor's field of view. Indeed, the LIDAR is mounted at a fixed height on the robot, meaning it can only detect obstacles that exist at the same height level.
This limitation results in some objects being partially or completely missed in the occupancy grid, leading to inaccuracies. Some notable cases include:

- Table legs with wide bases: in one of the rooms, a table with a widened leg base is present. While the tabletop is not an obstacle, the widened base is not detected because it is lower than the LIDAR sensor.
- Low furniture bases: all the shelves in the house have bases that remain too low to be detected by the robot's LIDAR.

Some figures highlighting these mapping errors are here reported, showing how some obstacles are missing from the final occupancy grid due to LIDAR height constraints.

Figure 2.3: Wide base table

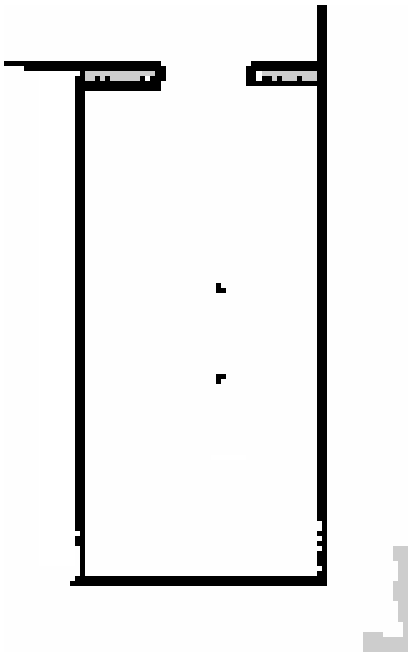

Figure 2.4: Missing table base

Figure 2.5: Shelf

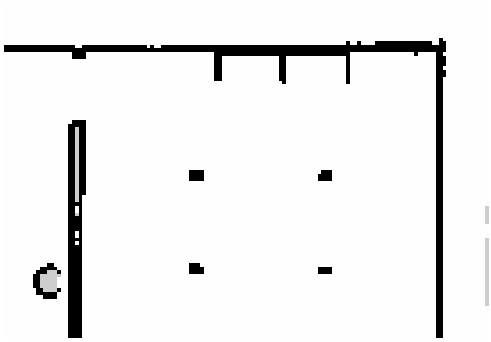

Figure 2.6: Missing shelf base

Obviously, these discrepancies between the real environment and the mapped one can lead to path planning issues. Indeed, if the robot perceives certain areas as clear and walkable, when in reality they are occupied by obstacles, the robot could plan paths that lead it into occupied spaces, resulting in collisions or errors.
For this reason, this discrepancy between the two environments will be taken into account and compensated for during the development of the pathfinding algorithm.

Chapter 3

A* algorithm implementation

In this project, the implementation of the A* algorithm for path planning has been significantly optimized to handle larger maps and improve computational efficiency. Several modifications were made compared to previous implementations, primarily focusing on optimizing memory usage and computation time. Below, the steps taken to implement and optimize the A* algorithm are detailed.

3.1 Map representation in MATLAB

Before diving into the A* algorithm implementation, the map generated through the SLAM process was converted into a format suitable for MATLAB. Indeed, the original occupancy grid map was first saved as a PNG file and then imported into MATLAB.
The map was subsequently converted into a boolean matrix, where each cell represented either a free space (1) or an occupied one (0), corresponding to navigable and non-navigable areas.
At this point, the missing elements in the map were manually adjusted to ensure that the map correctly included both the bases of the shelves and the wide base of the table legs.

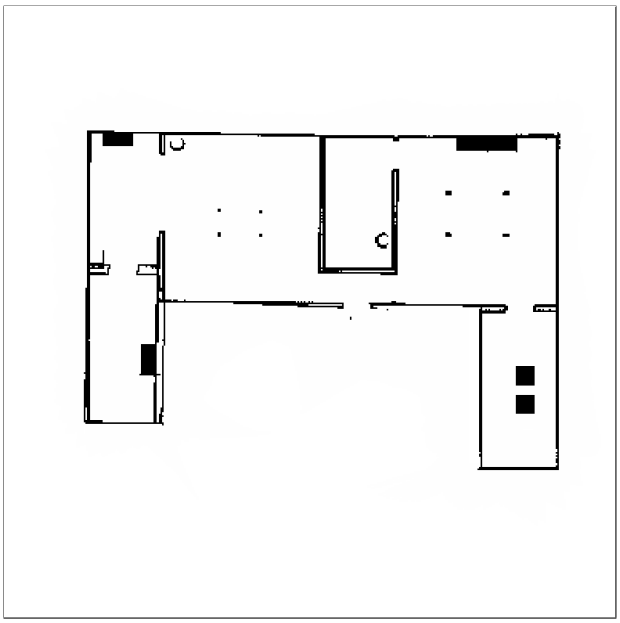

Figure 3.1: Corrected house map

To achieve this, the corresponding values in the boolean matrix were manually modified, knowing that the wide base of the table legs is a square with a side length of 0.55 m (measurement taken directly from Gazebo).

This matrix was then placed inside a larger 501x501 matrix, which allowed the origin of the coordinate system in Gazebo to coincide with the center of the matrix.
This step simplified the conversion between pixels and real-world coordinates during path planning.
Obviously, the scale factor between pixels and Cartesian coordinates should ideally be exactly 5 cm per pixel, based on the `gmapping` settings.
However, to account for small distortions and potential stretching present in the map, it was decided to calculate the scale factor by comparing the length of the right vertical side of the map with the known length of the corresponding side of the house in Gazebo (known dimension).
This approach allowed the axes to be rescaled, enabling everything to be plotted in terms of the Gazebo reference system from now on.

Furthermore, a safety zone around obstacles was defined. Indeed, any area within 0.105 m of an obstacle must be classified as inaccessible, ensuring that the robot, which has a certain physical width, would not come too close to any obstacles.

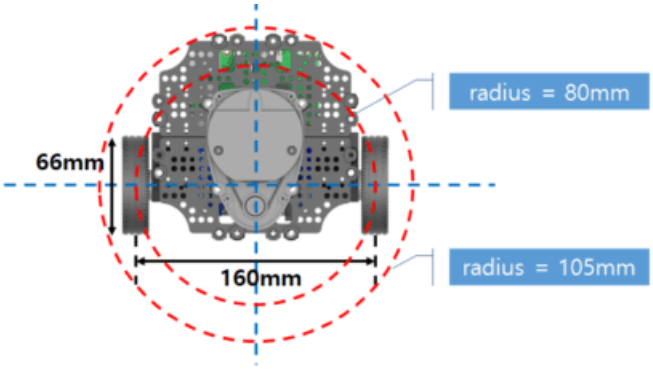

Figure 3.2: Turtlebot 3 dimensions

To account for this, it would have been sufficient to interpret as safety area all pixels whose distance from an obstacle was less than or equal to 3 pixels.
However, for greater precaution, it was decided to consider all pixels within a distance of 6 pixels.
This modification was essential for creating a realistic and safe path planning environment for the TurtleBot3 robot.
The following final complete map was therefore obtained:

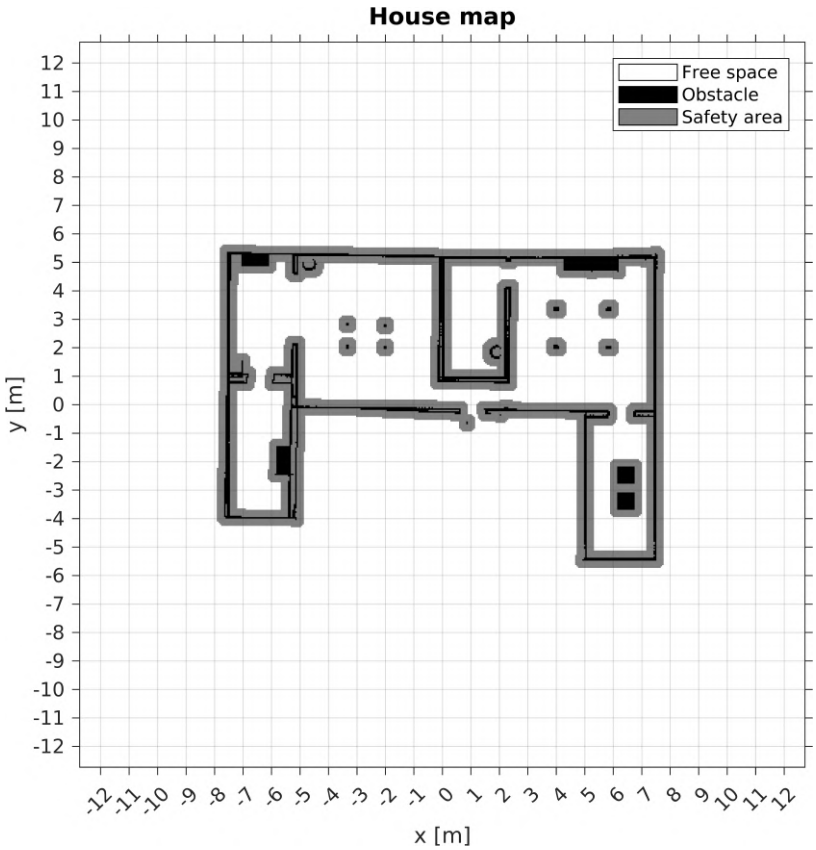

Figure 3.3: House map with enlarged obstacles

3.2 Optimizations in the A* algorithm

The A* algorithm was optimized to improve its performance for large maps, which previously presented computational challenges, especially with respect to memory usage and processing time.
In earlier versions of the algorithm, a large N×N matrix G, where N was the product of the map's dimensions, was used to store the costs of traversing between each pair of nodes.
This matrix, due to its size, introduced substantial computational overhead, scaling exponentially with the number of nodes. Indeed, in this project G would be of size (501x501)x(501x501), being obviously computationally expensive and impractical.
Additionally, various vectors and matrices were used in the code without considering that they could be converted easily into sparse matrices, thereby enhancing computational efficiency.
As such, the following optimization techniques were introduced.

3.2.1 Elimination of the full cost matrix

The most significant computational issue with previous implementations was the use of the full matrix G, where each element represented the traversal cost from one node to another.
This matrix was used solely to identify which nodes were connected and to calculate the cost to reach those nodes.
Thus, instead of using this large matrix, a function called `find_con_nodes` was developed. This function, when applied to any given node, returns a vector containing the indices of the neighboring nodes that are connected to it, excluding those that are either occupied or too close to obstacles.

This change drastically reduced the computational complexity of the algorithm, as it avoided the creation and manipulation of a massive matrix.

3.2.2 Introduction of sparse matrices

Another major optimization involved the use of sparse matrices. In the previous approach, at the beginning of the algorithm three key vectors were initialized: `dist`, `prec`, and `nodelist`.
These vectors were used to store the shortest distance from the start node to each node (`dist`), the predecessor of each node in the optimal path (`prec`), and the analysis state of each node (`nodelist`).
These vectors were initialized with infinity values and then updated iteration after iteration.
Notice that in the `nodelist` vector:

- `0` : indicates that a certain node has already been analyzed.
- `1` : indicates that a certain node has to be analyzed in the future.
- `inf` : indicates that a certain node has never been considered yet.

After several trials, given that most of the nodes in the grid were not visited during the executions of the algorithm (compared to the total number of nodes), the algorithm values logic was modified to initialize these vectors as sparse matrices.
This reduced memory usage and improved the computational efficiency of the code.
Specifically, the vectors were initialized as sparse matrices, with their values redefined as follows:

- `-1` : indicates that the node has already been analyzed;
- `1` : indicates that the node will be analyzed in the future.
- `0` : indicates that the node has not been considered;

This change allowed for faster updates and better memory management, as only the nodes that were actively being analyzed or had been visited were stored.

3.2.3 Sparse representation of the map

Additionally, the map representation itself was modified to improve efficiency. Indeed, the boolean matrix `BW` representing the environment was replaced with a sparse matrix, `BW_sparse`, in which 0 and 1 values were now inverted. Thus, such sparse matrix stored only the non accessible areas of the map, being more efficient for the algorithm use.
The original matrix `BW` instead was used just for graphical purposes, such as displaying the grid and obstacles in visualization figures.
This sparse representation was crucial for reducing memory consumption, particularly when dealing with larger environments. By representing only the non-zero values in a compact format, the algorithm was able to work more efficiently without storing redundant information.

3.3 Algorithm considerations

To conclude, in the previous implementation, the map was visualized using the `plot_map` function, which iterated over each element of the matrix (501×501 iterations) and plotted each point individually within the same figure.
Although functional, this approach was computationally expensive for visualization purposes.
To optimize the visualization process, the `imshow` function was used instead.
This method allows for direct rendering of the entire matrix in a single step, significantly reducing computational overhead.
However it is important to note that this optimization strictly improves visualization performance

and does not impact the actual execution efficiency of the A* algorithm, as it is only related to how the results are displayed rather than how they are computed.

Finally, to provide a clearer and more comprehensive overview of the developed algorithm, it is important to explicitly state that the chosen heuristic function is the diagonal distance between the considered point and the goal.
This distance was selected to remain consistent with the allowed movement types, which include only the four cardinal directions and the four main diagonals.
In cases where two connected nodes have the same cost, preference is given to the node with the smallest diagonal distance to the goal.
This heuristic function ensures the proper functioning of the A* algorithm, maintaining its ability to always find the shortest path when one exists.

To conclude, the optimized A* algorithm was tested on several start-end point pairs within the mapped environment. The algorithm successfully computed the shortest paths between the designated start and goal points while avoiding obstacles and adhering to the safety zones.
Despite the significant improvements in computational efficiency, the algorithm still has two important limitations:

- Prior knowledge of the map: the A* algorithm requires a pre-known map to plan the path. This makes it less suitable for unknown or partially explored scenarios.
- Assumption of a static map: the algorithm assumes that the map remains unchanged during path planning and path tracking. If new obstacles are detected after the path has been computed, the robot may not be able to adapt in real time unless a dynamic re-planning mechanism is implemented.

However, the optimized A* algorithm has proven to be effective for pathfinding in such a map, significantly improving computational efficiency through the use of sparse matrices and the elimination of unnecessary cost matrices.
These optimizations have allowed the algorithm to handle larger environments while keeping processing times reasonable.

Chapter 4

Pathfinding results and algorithm results visualization

This chapter presents the results of the pathfinding algorithm applied to different start and goal point pairs.
Each section includes a map showing the start and goal points, a visualization of the nodes considered during the search, and the identified optimal path.
Additionally, performance metrics such as the path length, computation time, and the number of nodes evaluated are provided.
These results demonstrate the algorithm's effectiveness across a variety of test cases.

4.1 Path 1 calculation

Chosen start and goal points:

- Start: (-7 m , -3 m)
- Goal: (1 m , 2 m)

The first pathfinding test shows the algorithm's performance for the specified start and goal locations. The following results are obtained:

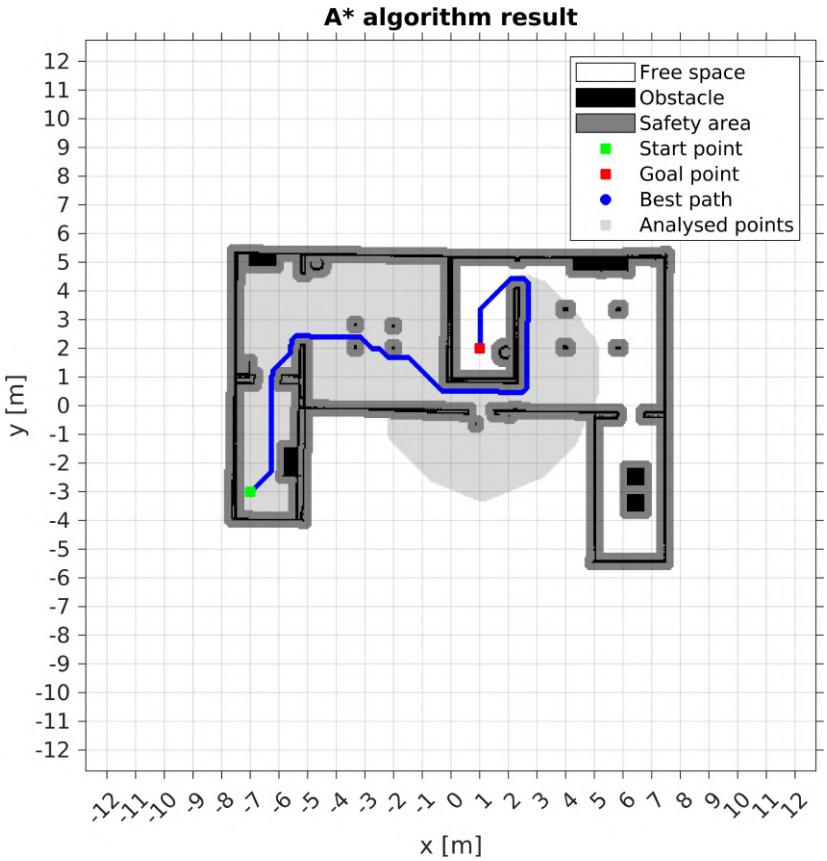

Figure 4.1: A* algorithm results for path 1

Technical results:

Metric	Value
Start point	(-7 m , -3 m)
Goal point	(1 m , 2 m)
Path length	22.2893 m
Number of analysed nodes	21222
Algorithm required time	6.4558 s

Table 4.1: Technical results for path 1

4.2 Path 2 calculation

Chosen start and goal points:

- Start: (-10 m , 4 m)
- Goal: (6 m , -4.5 m)

The second pathfinding test shows the algorithm's performance for the specified start and goal locations. The following results are obtained:

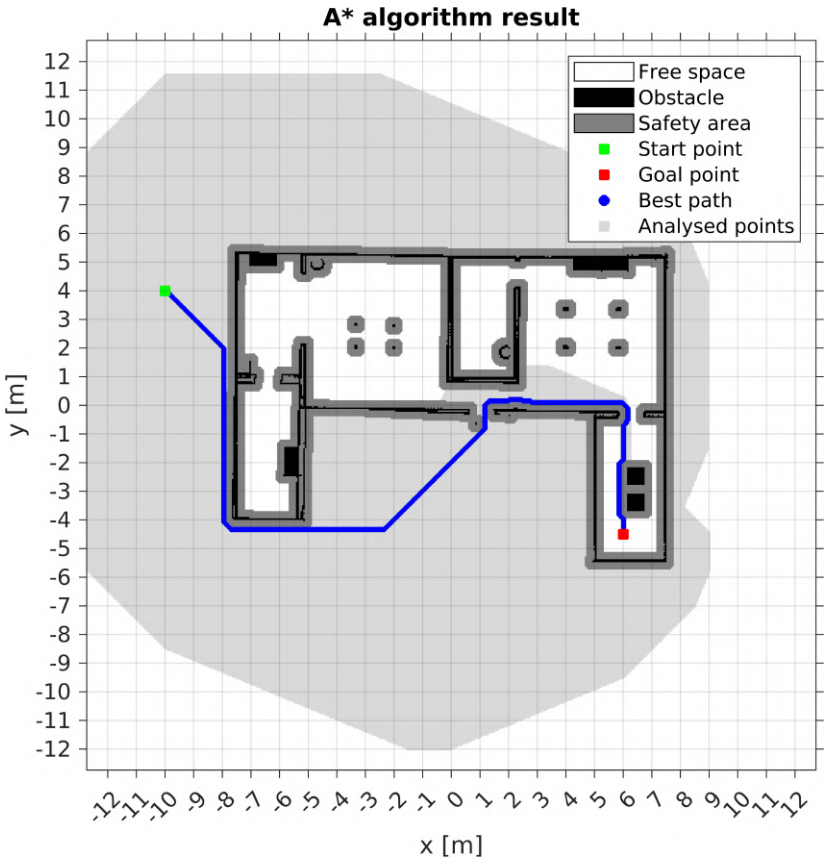

Figure 4.2: A* algorithm results for path 2

Technical results:

Metric	Value
Start point	(-10 m , 4 m)
Goal point	(6 m , -4.5 m)
Path length	30.2098 m
Number of analysed nodes	121106
Algorithm required time	194.0913 s

Table 4.2: Technical results for path 2

4.3 Path 3 calculation

Chosen start and goal points:

- Start: (10 m , 2 m)
- Goal: (-7 m , 3 m)

The third pathfinding test shows the algorithm's performance for the specified start and goal locations. The following results are obtained:

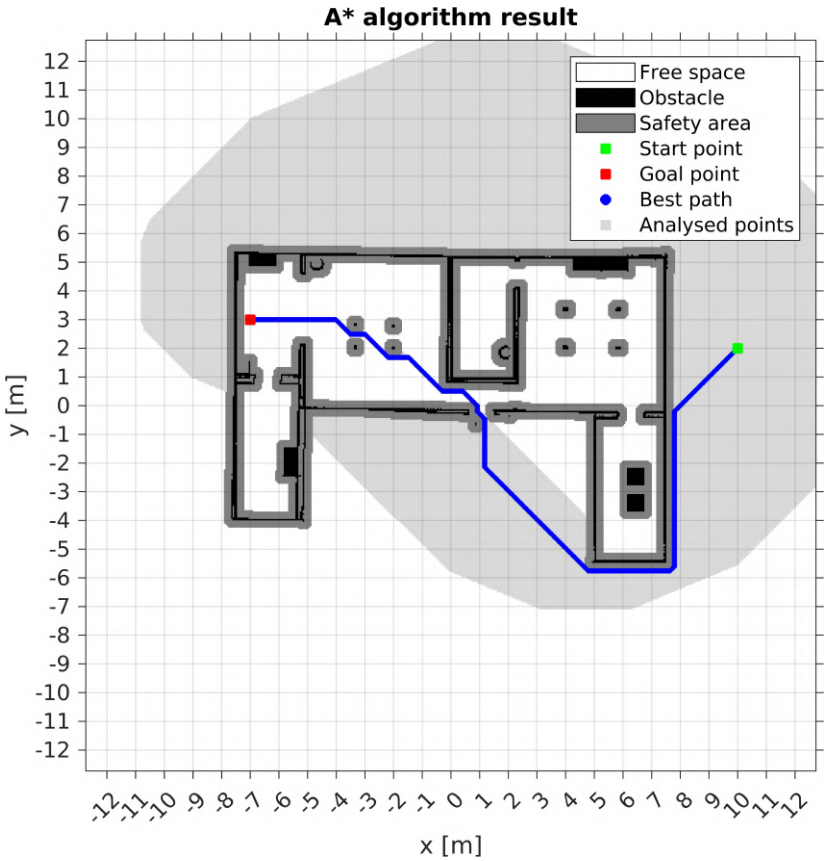

Figure 4.3: A* algorithm results for path 3

Technical results:

Metric	Value
Start point	(10 m , 2 m)
Goal point	(-7 m , 3 m)
Path length	28.0645 m
Number of analysed nodes	89018
Algorithm required time	100.4219 s

Table 4.3: Technical results for path 3

4.4 Path 4 calculation

Chosen start and goal points:

- Start: (0 , 10)
- Goal: (-6 , -1)

The fourth pathfinding test shows the algorithm's performance for the specified start and goal locations. The following results are obtained:

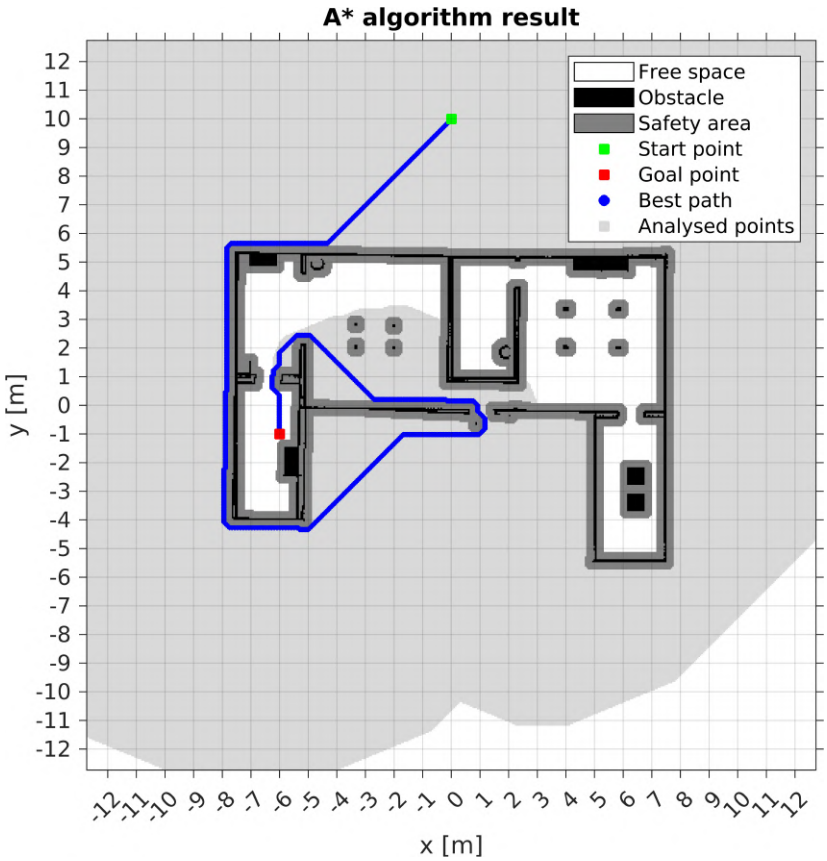

Figure 4.4: A* algorithm results for path 4

Technical results:

Metric	Value
Start point	(0 , 10)
Goal point	(-6 , -1)
Path length	42.175 m
Number of analysed nodes	190829
Algorithm required time	354.6933 s

Table 4.4: Technical results for path 4

4.5 Path 5 calculation

Chosen start and goal points:

- Start: (6.8 , -5)
- Goal: (-10 , 3)

The following pathfinding test shows the algorithm's performance for the specified start and goal locations. The following results are obtained:

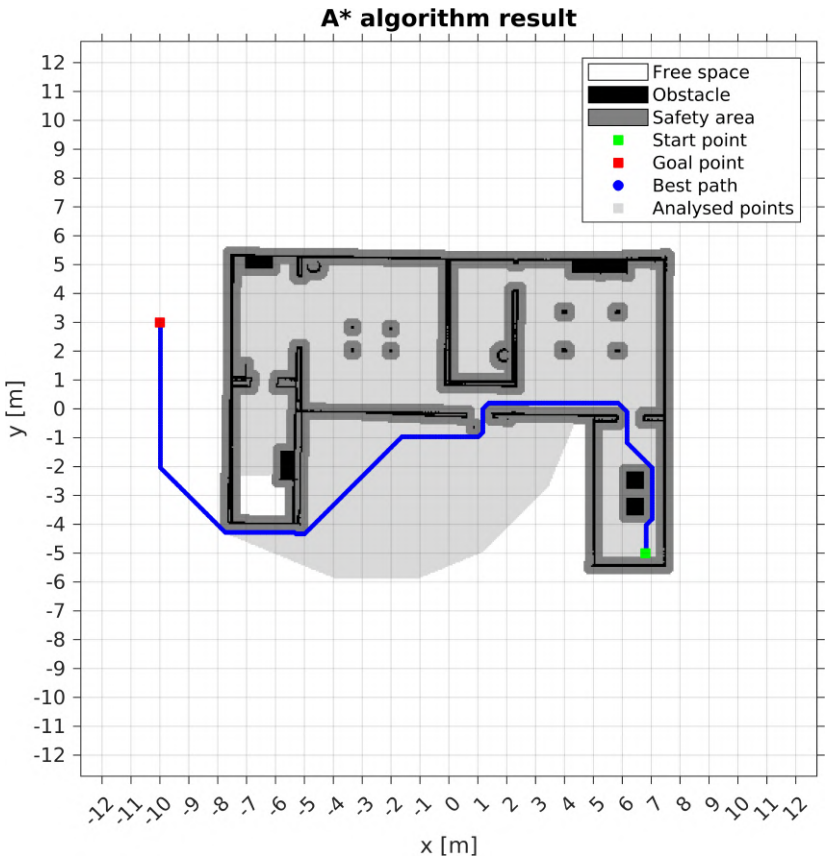

Figure 4.5: A* algorithm results for path 5

Technical results:

Metric	Value
Start point	(6.8 , -5)
Goal point	(-10 , 3)
Path length	29.9536 m
Number of analysed nodes	40136
Algorithm required time	19.3465 s

Table 4.5: Technical results for path 5

4.6 Path 6 calculation

Chosen start and goal points:

- Start: (0.5 , 2)
- Goal: (11 , -8)

The following pathfinding test shows the algorithm's performance for the specified start and goal locations. The following results are obtained:

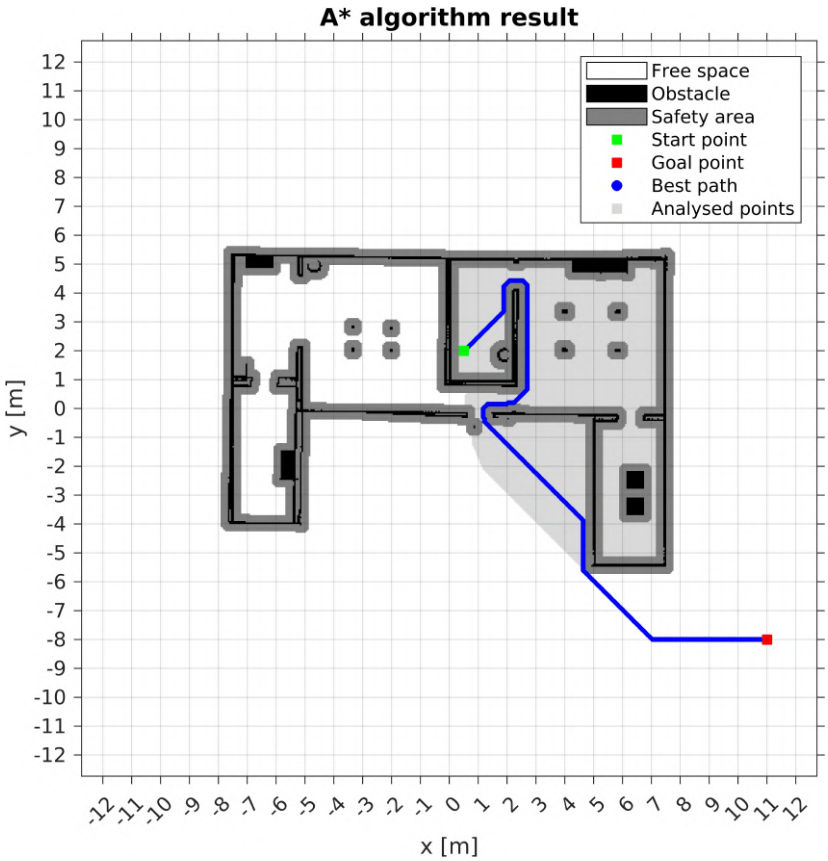

Figure 4.6: A* algorithm results for path 6

Technical results:

Metric	Value
Start point	(0.5 , 2)
Goal point	(11 , -8)
Path length	23.5887 m
Number of analysed nodes	16970
Algorithm required time	5.3653 s

Table 4.6: Technical results for path 6

Chapter 5

Feedback control for trajectory tracking

This section presents the implementation of the feedback control system for trajectory tracking, which serves as an additional enhancement to the project.
The control system was designed and simulated in Simulink, leveraging ROS integration to enable real-time tracking of the robot's position and ensure smooth and accurate trajectory following.

5.1 Control system architecture

The control framework was developed in Simulink with ROS blocks to facilitate real-time communication with the robot.
The foundation of the system, as shown in the following figure, is based on a subsystem called Feedback Control, which outputs the linear velocity (LinVel) and angular velocity (AngVel) required to guide the robot along the desired trajectory.
These values are first used to populate the blank message created in Simulink and then directly published to the `/cmd_vel` topic using a ROS Block `Publisher`.
This block ensures that the correct message is sent to the appropriate topic, carrying the two output signals from the Feedback Control subsystem.

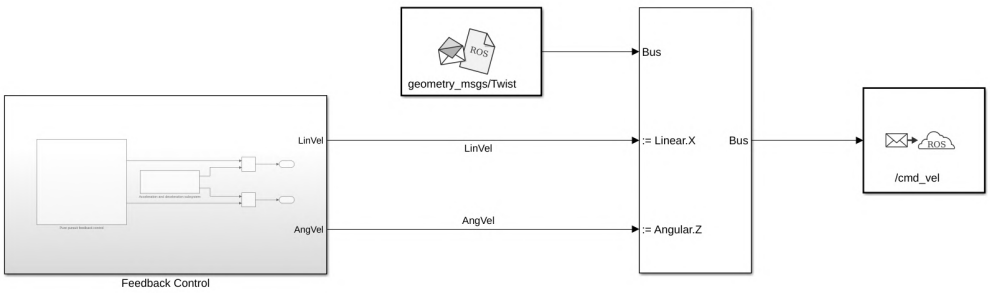

Figure 5.1: Block diagram of the Simulink main system

Let's now take a closer look at how the Feedback Control subsystem operates.
As shown in Figure 5.2, inside the subsystem there is a Pure pursuit feedback control subsystem, which is the actual Simulink subsystem responsible for determining the linear velocity and angular velocity signals required for path tracking.

However, it is important to note that these signals are each multiplied by a gain generated by an additional subsystem, which ensures smooth initial acceleration and smooth final deceleration.

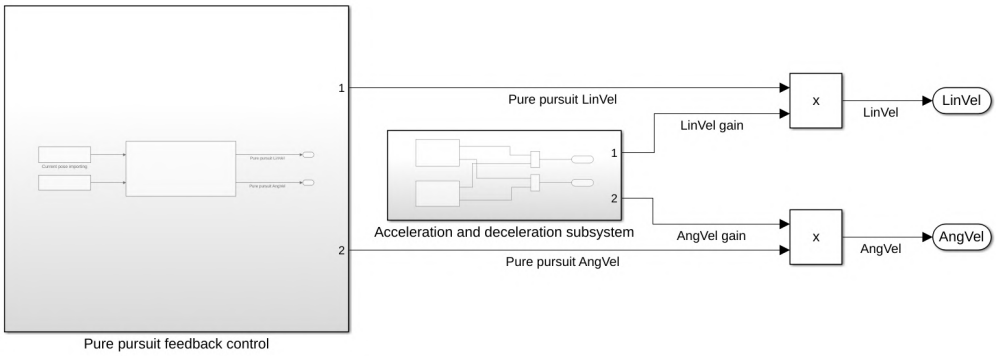

Figure 5.2: Feedback control subsystem block diagram

Focusing now on the Pure Pursuit feedback control subsystem, it can be seen from Figure 5.3 that the Pure Pursuit block was used.
This block takes as input the current position of the robot (imported into Simulink using the ROS block `Subscriber`), and a two-column matrix where:

- the first column contains the x coordinates of the optimal path computed in MATLAB;
- the second column contains the corresponding y coordinates of the path.

The outputs of the block are simply the two velocities required to ensure correct path tracking.

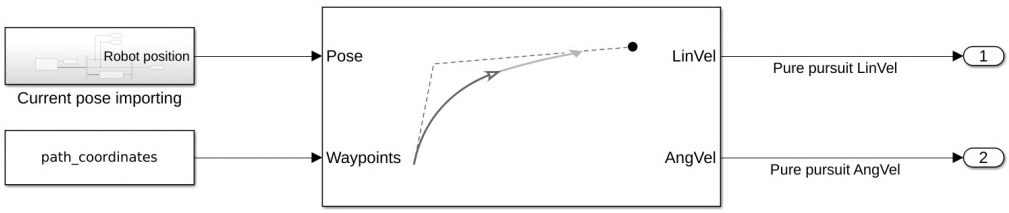

Figure 5.3: Pure pursuit feedback control subsystem block diagram

After extensive testing, the following Pure pursuit block control parameters were selected to achieve stable and precise trajectory tracking:

- Desired linear velocity: 0.12 m/s;
- Maximum angular velocity: 0.35 rad/s;
- Lookahead distance: 0.25 m.

These values were chosen to balance responsiveness and smoothness, ensuring that the robot followed the trajectory efficiently without excessive oscillations or deviations.

To further improve the system's performance, as mentioned before, an additional subsystem called Acceleration and Deceleration subsystem was introduced.
Inside this subsystem, there are two subsystems: one aimed at ensuring smooth acceleration during

the initial moments of the simulation, and another designed to guarantee smooth deceleration as the robot approaches the goal position.

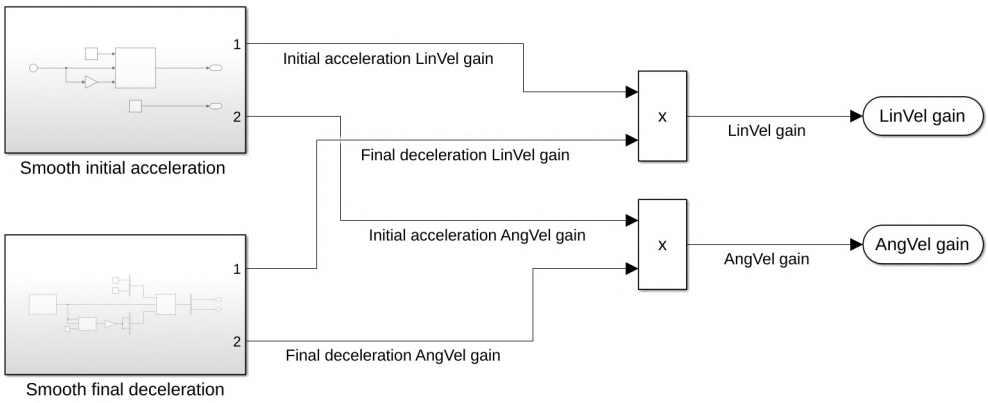

Figure 5.4: Acceleration and deceleration subsystem block diagram

5.1.1 Initial acceleration control

The first subsystem is designed to ensure a gradual acceleration during the first few seconds of simulation.
In accordance with the system's architecture, during the first two seconds of motion, the system gradually increases the velocity from zero to the desired speed (0.12 m/s).
It is important to note that this is achieved by multiplying the signal calculated by the Pure Pursuit Feedback Control subsystem with the gains output from this subsystem, which properly scales the velocity during the initial seconds of simulation to reach the desired velocity profile.

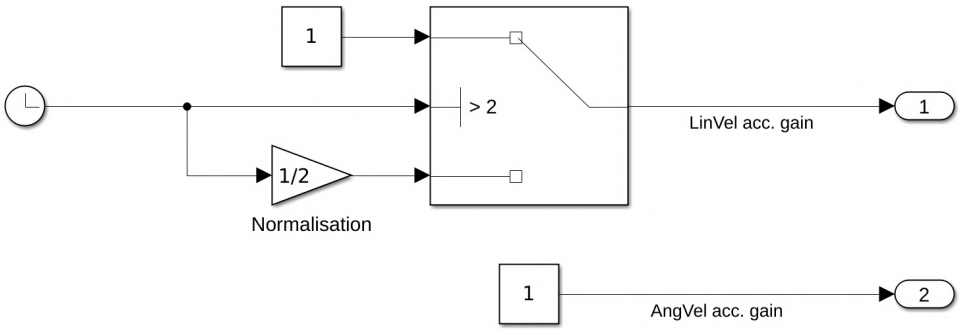

Figure 5.5: Smooth initial acceleration subsystem block diagram

Gradual acceleration during the first seconds of motion prevented abrupt starts improving stability. Additionally, this was necessary because if the robot starts with an incorrect orientation and a high linear velocity, it would move far away from the trajectory reference while rotating during the first few seconds, thus increasing the risk of collisions with obstacles.
Instead, by starting with a lower initial velocity, the robot advances slowly while adjusting its orientation, minimizing trajectory deviation and ensuring a smoother transition into the path.

5.1.2 Final deceleration control

The second subsystem is designed to ensure proportional deceleration near the goal.
As the robot approaches the target position, the velocity is progressively reduced in proportion to the remaining distance. This approach ensures that the robot properly stops at the goal.
Additionally, a tolerance of 3 cm was introduced, within which the robot is programmed to stop once it enters a 3 cm radius from the goal.
Without this measures, the robot would reach the goal but then diverging from that position, exhibiting highly unstable behavior that caused it to repeatedly move away from and then return close to the goal.
By imposing the stopping condition within this tolerance, the system ensures the robot comes to a stop within an acceptable range, preventing oscillation around the goal.

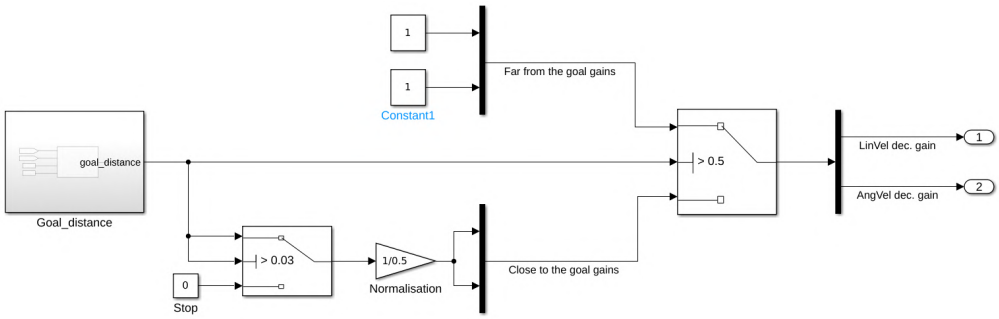

Figure 5.6: Smooth final deceleration subsystem block diagram

5.2 Control system conclusion

The implementation of the feedback control system in Simulink demonstrated effective trajectory tracking, enabling the robot to follow the computed path accurately while maintaining smooth and stable motion.
The integration of real-time odometry feedback and the tuning of control parameters contributed to the overall success of the approach.

Chapter 6

Trajectory tracking results

Once the control system was correctly implemented, the best paths calculated in chapter 4 were tested by collecting the data published on the `/odom` topic by Gazebo into a bag file.
This allowed the effectiveness of the control system to be evaluated by comparing the reference path with the actual trajectory followed by the robot.
Additionally, the velocity profiles followed by the robot were visualized to further assess the performance of the control system.
Before diving into the results, let's briefly explain how the time required by the Gazebo simulation to complete the path was estimated.

6.1 Time estimation for Gazebo simulation

The time required for the Gazebo simulation to complete the path was estimated by dividing the trajectory into three distinct phases:

- Acceleration phase:
  The first phase lasts for 2 seconds (Δt_1), during which the robot accelerates according to the following motion law:
  $$v(t) = v_{\text{des}} \cdot \frac{t}{2}$$
  where $v_{\text{des}} = 0.12\,\text{m/s}$.
  In this phase, the total distance s_1 traveled, after 2 seconds can be calculated as:
  $$s_1 = \int_0^2 v(t)\,dt = \int_0^2 0.12 \cdot \frac{t}{2}\,dt = 0.12 \cdot \frac{1}{2} \cdot \frac{t^2}{2}\bigg|_0^2 = 0.03 \cdot 2^2 = 0.12 \text{ m}$$
  So, the distance traveled during the first 2 seconds is $s_1 = 0.12\,\text{m}$.

- Constant velocity phase:
  In the second phase, the robot continues traveling a distance equal to $s_{\text{tot}} - 0.5 - s_1$, at a constant speed of 0.12 m/s.
  Thus, the time taken to travel this distance is given by:
  $$\Delta t_2 = \frac{s_{\text{tot}} - 0.5 - s_1}{v_{\text{des}}} = \frac{s_{\text{tot}} - 0.62}{0.12} \text{ s}$$

- Deceleration phase:
  In the final phase, the robot travels a distance of 0.47 m, starting from a velocity of 0.12 m/s.

The velocity decreases proportionally to the distance traveled, reaching 0 m/s when the robot has traveled 0.5 m. The velocity function in this phase can be described as:

$$v(x) = v_{\text{des}} \cdot \left(1 - \frac{x}{0.5}\right)$$

where x is the distance traveled in this phase.
The time taken to travel the remaining 0.47 m is obtained by integrating the inverse of the velocity:

$$\Delta t_3 = \int_0^{0.47} \frac{1}{v(x)}\, dx = \int_0^{0.47} \frac{1}{0.12 \cdot \left(1 - \frac{x}{0.5}\right)}\, dx$$

This integral can be analytically solved to obtain the time for the third phase:

$$\Delta t_3 = \frac{1}{0.12} \int_0^{0.47} \frac{1}{1 - \frac{x}{0.5}}\, dx = \frac{1}{0.12} \left[-0.5 \ln\left(1 - \frac{x}{0.5}\right)\right]_0^{0.47} =$$

$$= \frac{1}{0.12} \left[-0.5 \ln\left(1 - \frac{0.47}{0.5}\right)\right]_0^{0.47} = -\frac{100}{24} \ln 0.06 = \frac{25}{6} \ln \frac{50}{3} \approx 11.72 \text{ s}$$

The estimation of the total time required to complete the path is then the sum of the times for all three phases:

$$T_{\text{tot}} = \Delta t_1 + \Delta t_2 + \Delta t_3 = 2 + \frac{s_{\text{tot}} - 0.62}{0.12} + \frac{25}{6} \ln \frac{50}{3} \text{ s}$$

6.2 Path tracking results

In this section, the results of applying the developed control system to the optimal paths previously computed in chapter 4 are presented.
The focus is on comparing the expected results, which were estimated before running the simulation, with the actual outcomes obtained from the execution in Gazebo.
This analysis allows for an evaluation of the accuracy and effectiveness of the trajectory tracking system. Indeed, as shown in the following velocity profiles, the linear velocity remains consistently close to the desired value of 0.12 m/s, except during the initial and final transients.
In particular, the final phase exhibits the expected exponential decay, smoothly bringing the robot to a stop near the goal point.
At the same way, the angular velocity always stays within the imposed maximum limits of $\pm$ 3.5 rad/s, ensuring stable and controlled trajectory tracking throughout the execution.

6.2.1 Path 1 execution

In the next figure, the planned trajectory is overlaid with the executed trajectory in Gazebo within the simulated environment.

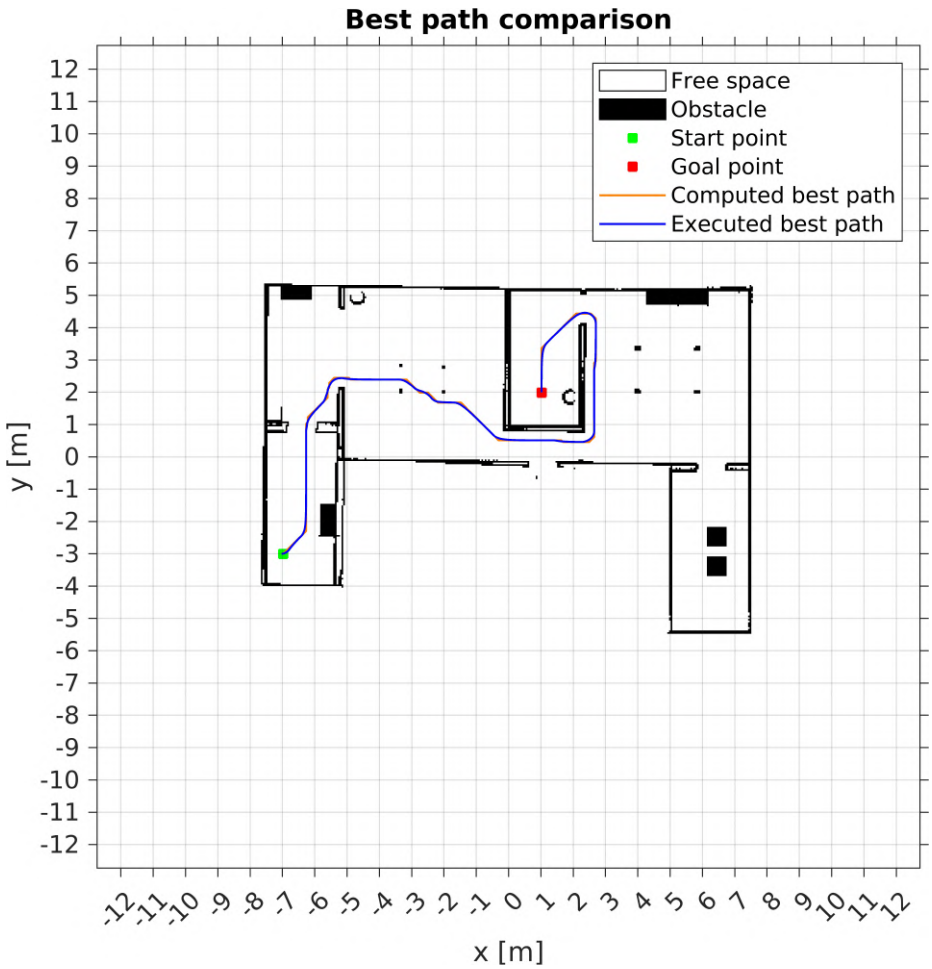

Figure 6.1: Path 1 Gazebo execution analysis

The following figure presents the time histories of the linear and angular velocities recorded during the simulation.

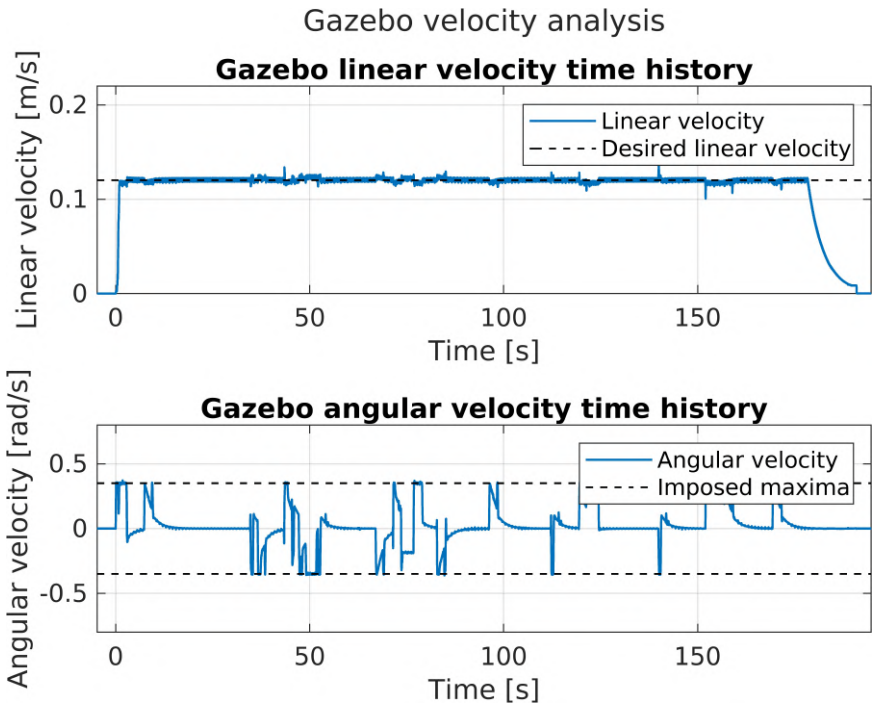

Figure 6.2: Path 1 Gazebo velocity analysis

Finally, the next table summarizes the key numerical results, providing a direct comparison between the expected and executed trajectory metrics.

Metric	**Value**
Expected path length	22.2893 m
Executed path length	21.7382 m
Expected travel time	194.3003 s
Executed travel time	190.0338 s

Table 6.1: Expected versus executed trajectory metrics for Path 1

6.2.2 Path 2 execution

In the next figure, the planned trajectory is overlaid with the executed trajectory in Gazebo within the simulated environment.

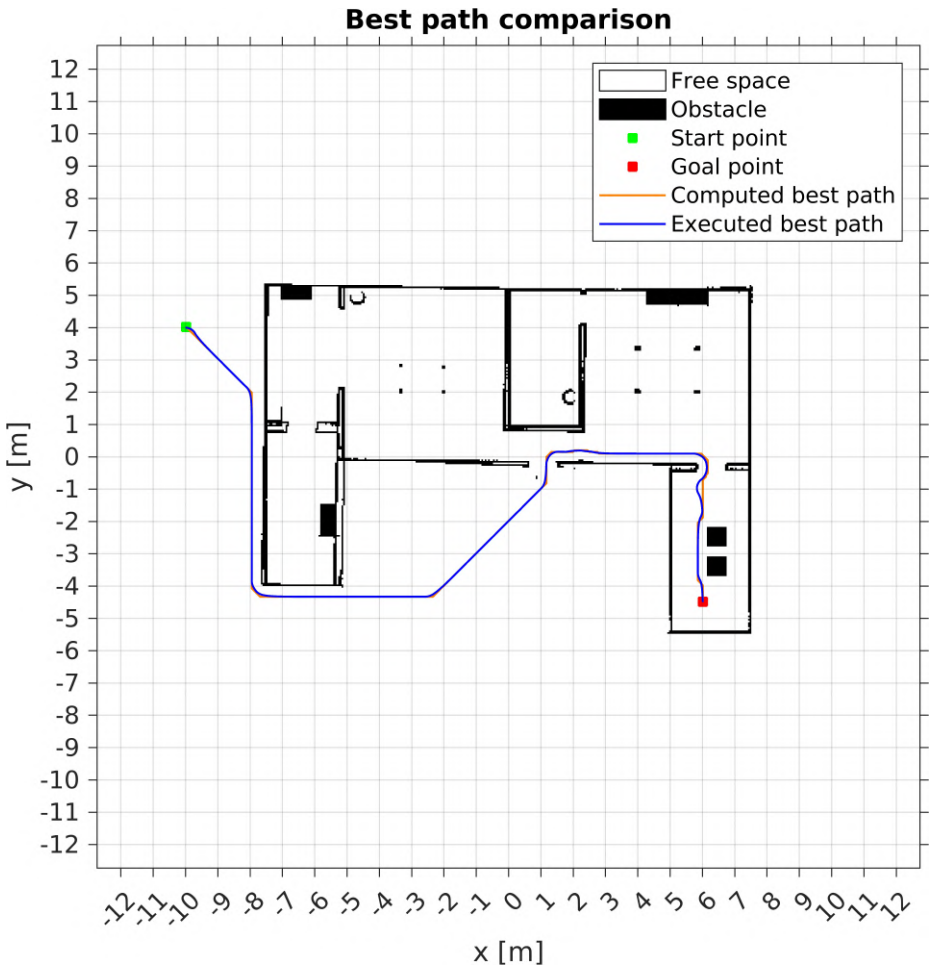

Figure 6.3: Path 2 Gazebo execution analysis

The following figure presents the time histories of the linear and angular velocities recorded during the simulation.

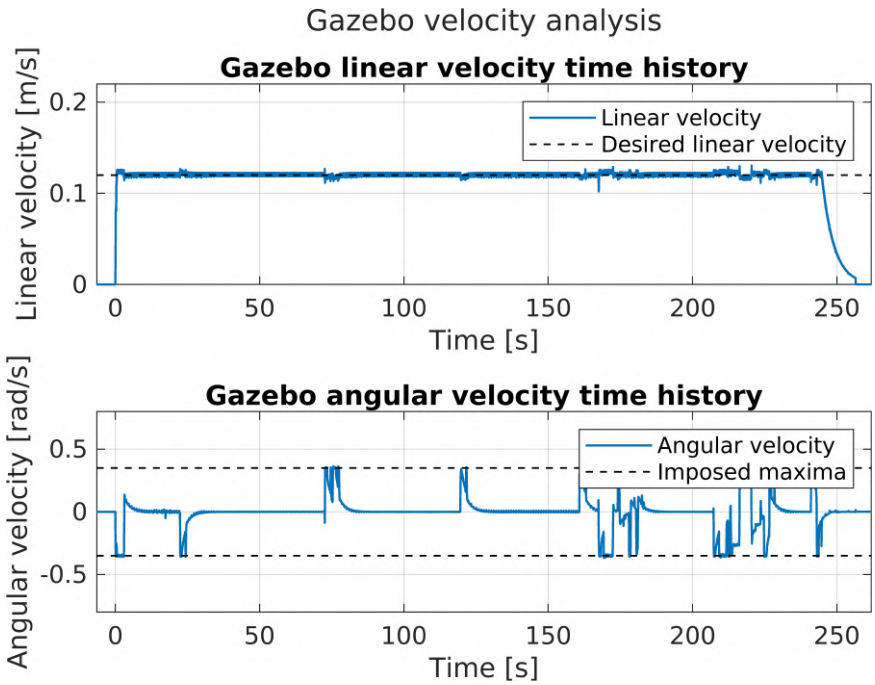

Figure 6.4: Path 2 Gazebo velocity analysis

Finally, the next table summarizes the key numerical results, providing a direct comparison between the expected and executed trajectory metrics.

Metric	**Value**
Expected path length	30.2098 m
Executed path length	29.2993 m
Expected travel time	260.3041 s
Executed travel time	255.5665 s

Table 6.2: Expected versus executed trajectory metrics for Path 2

6.2.3 Path 3 execution

In the next figure, the planned trajectory is overlaid with the executed trajectory in Gazebo within the simulated environment.

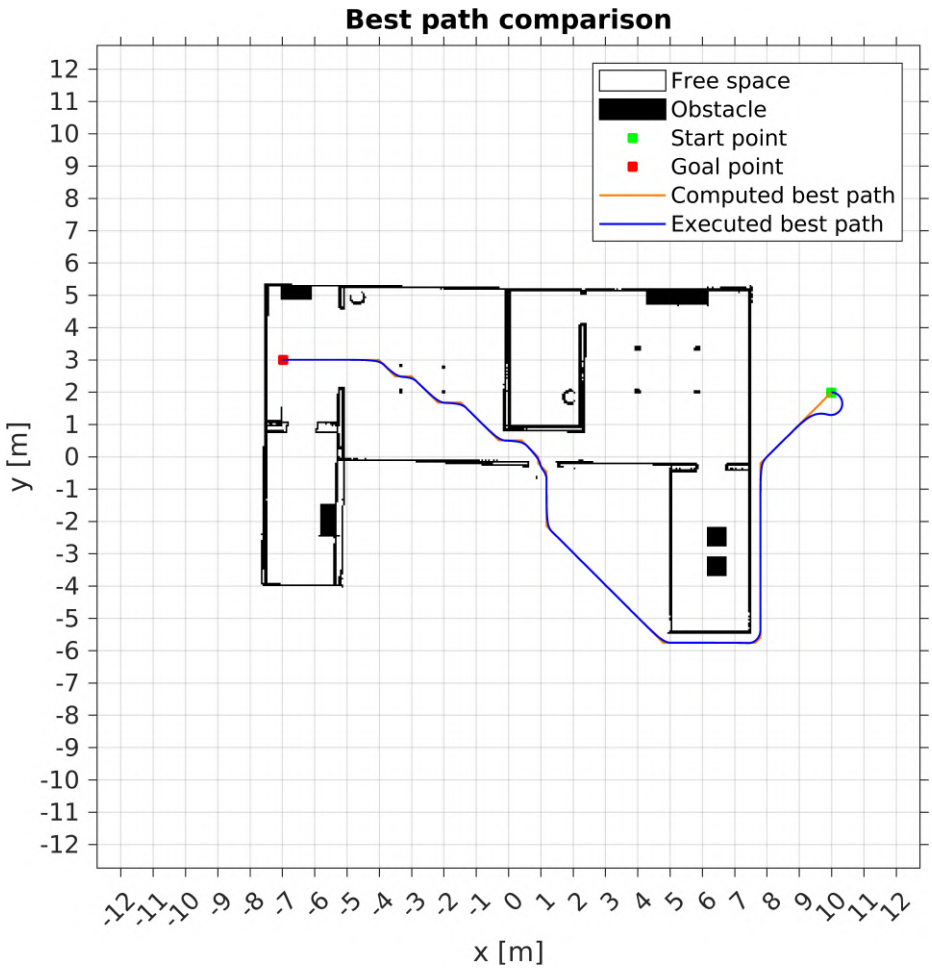

Figure 6.5: Path 3 Gazebo execution analysis

The following figure presents the time histories of the linear and angular velocities recorded during the simulation.

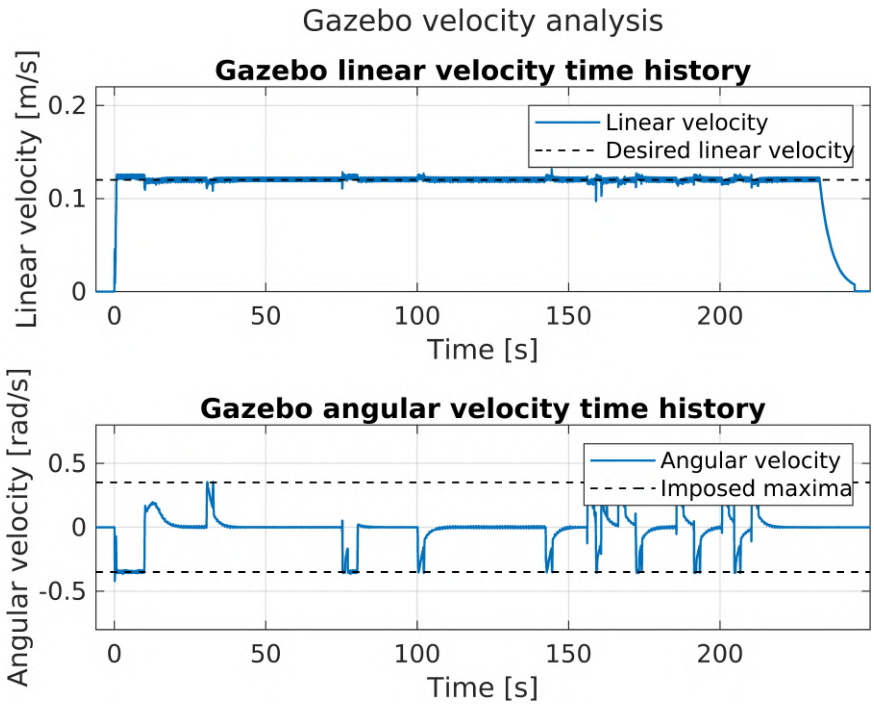

Figure 6.6: Path 3 Gazebo velocity analysis

Finally, the next table summarizes the key numerical results, providing a direct comparison between the expected and executed trajectory metrics.

Metric	**Value**
Expected path length	28.0645 m
Executed path length	28.3532 m
Expected travel time	242.4265 s
Executed travel time	243.566 s

Table 6.3: Expected versus executed trajectory metrics for Path 3

6.2.4 Path 4 execution

In the next figure, the planned trajectory is overlaid with the executed trajectory in Gazebo within the simulated environment.

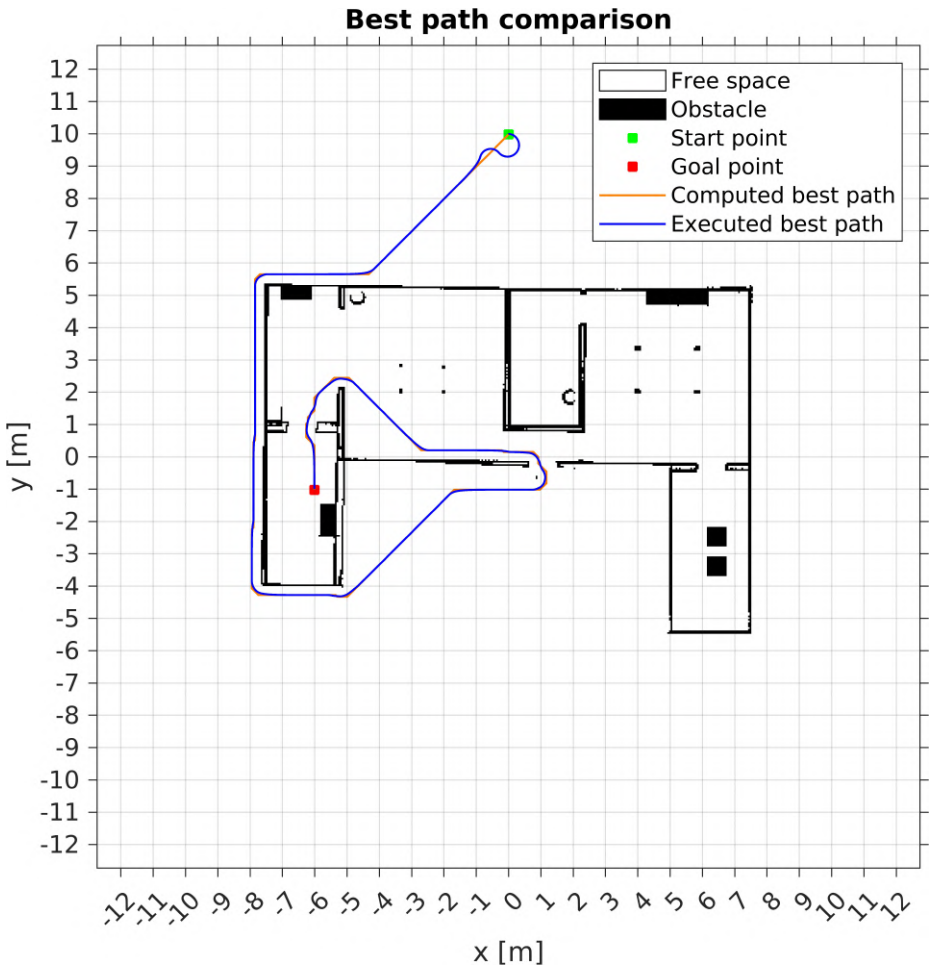

Figure 6.7: Path 4 Gazebo execution analysis

The following figure presents the time histories of the linear and angular velocities recorded during the simulation.

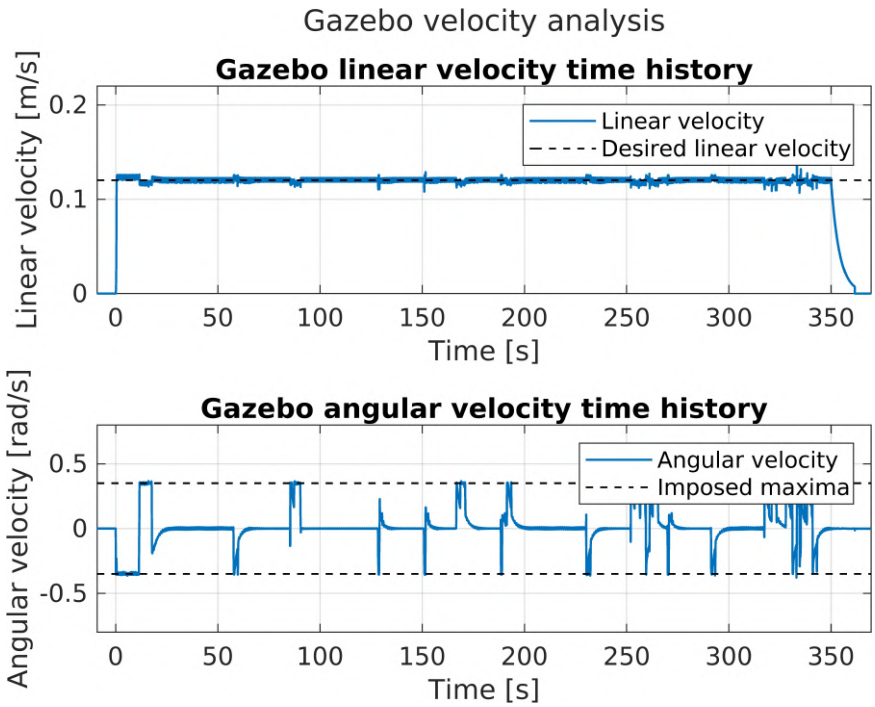

Figure 6.8: Path 4 Gazebo velocity analysis

Finally, the next table summarizes the key numerical results, providing a direct comparison between the expected and executed trajectory metrics.

Metric	**Value**
Expected path length	42.175 m
Executed path length	42.3301 m
Expected travel time	360.0146 s
Executed travel time	360.6665 s

Table 6.4: Expected versus executed trajectory metrics for Path 4

6.2.5 Path 5 execution

In the next figure, the planned trajectory is overlaid with the executed trajectory in Gazebo within the simulated environment.

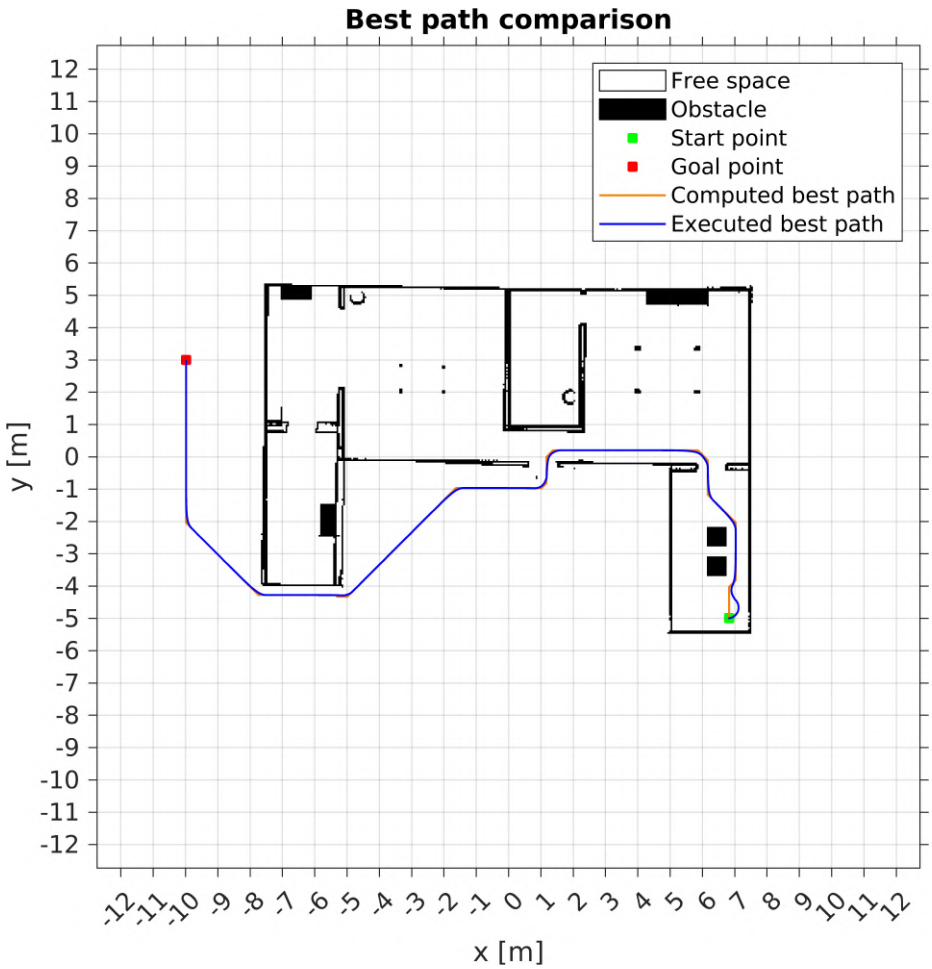

Figure 6.9: Path 5 Gazebo execution analysis

The following figure presents the time histories of the linear and angular velocities recorded during the simulation.

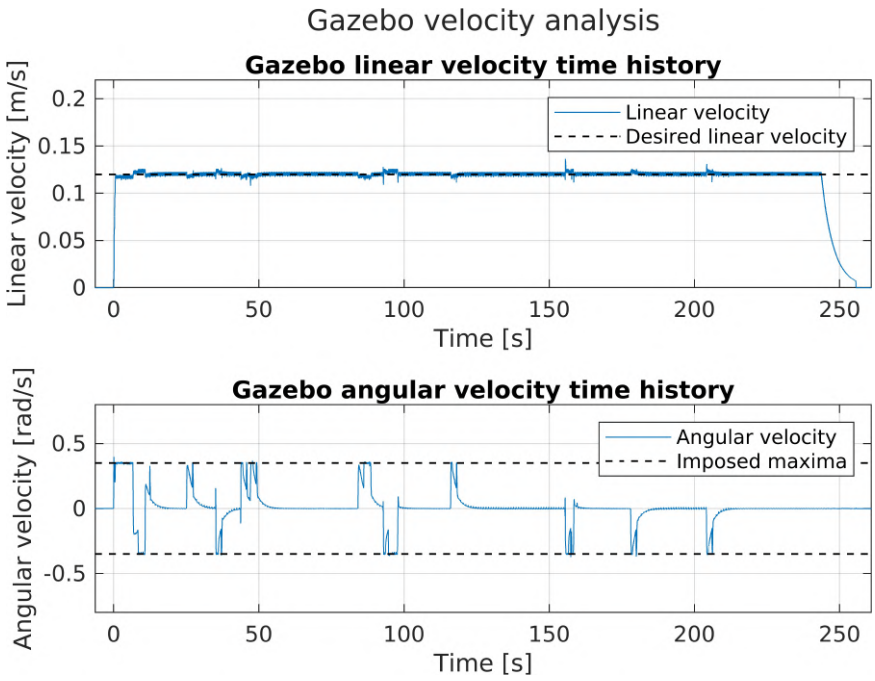

Figure 6.10: Path 5 Gazebo velocity analysis

Finally, the next table summarizes the key numerical results, providing a direct comparison between the expected and executed trajectory metrics.

Metric	**Value**
Expected path length	29.9536 m
Executed path length	29.6331 m
Expected travel time	258.1693 s
Executed travel time	254.6327 s

Table 6.5: Expected versus executed trajectory metrics for Path 5

6.2.6 Path 6 execution

In the next figure, the planned trajectory is overlaid with the executed trajectory in Gazebo within the simulated environment.

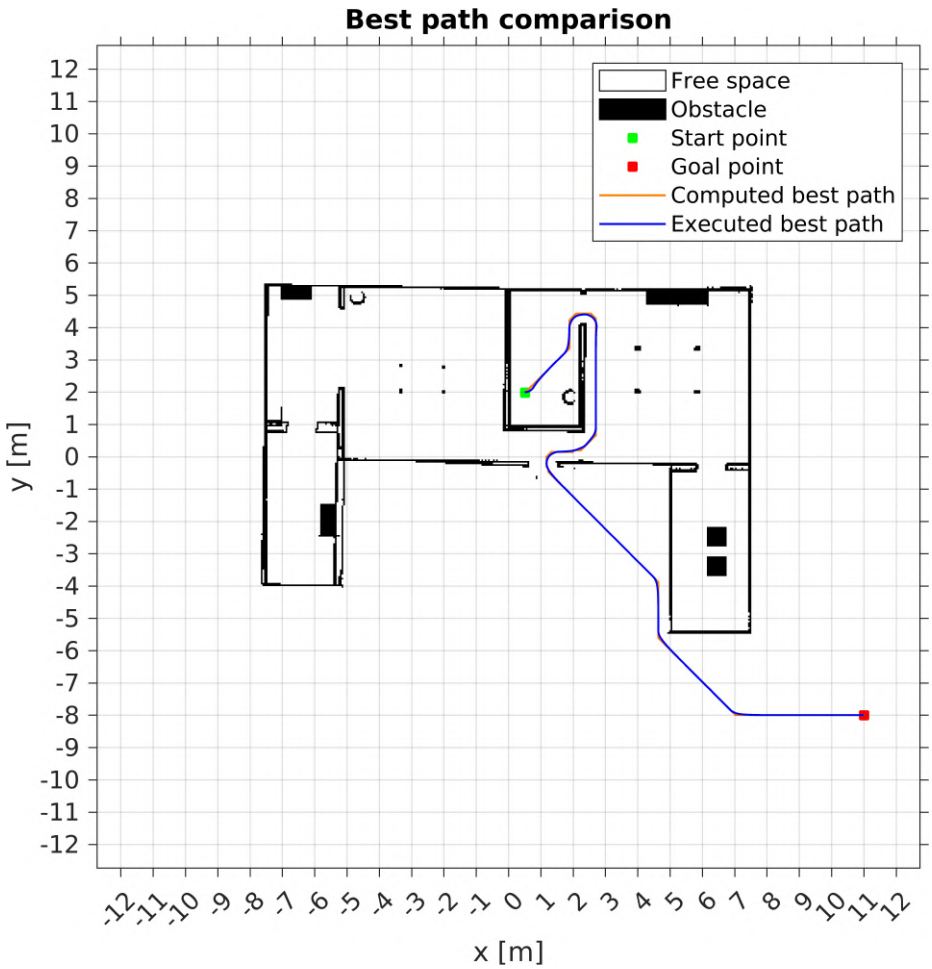

Figure 6.11: Path 6 Gazebo execution analysis

The following figure presents the time histories of the linear and angular velocities recorded during the simulation.

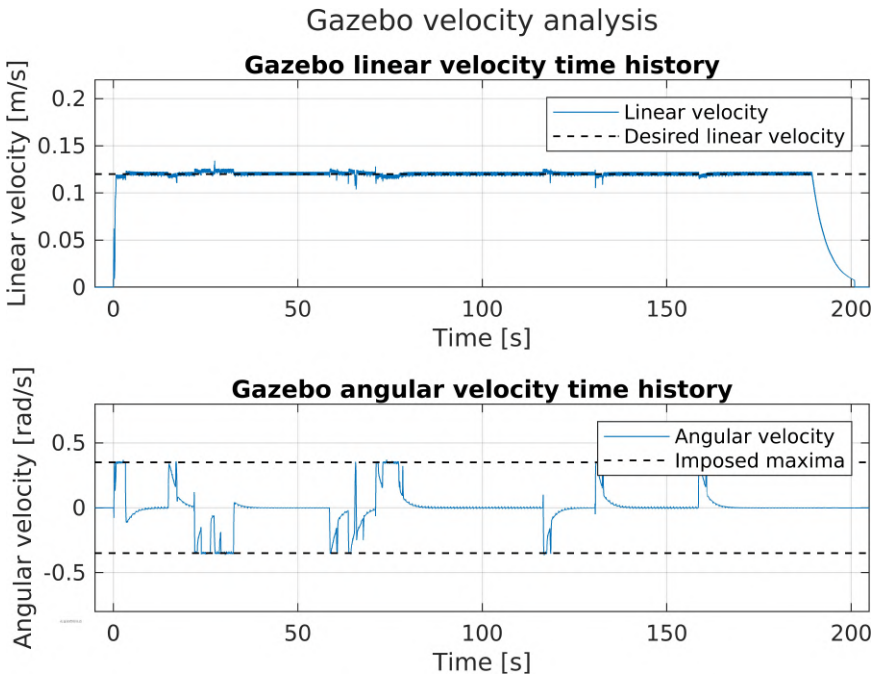

Figure 6.12: Path 6 Gazebo velocity analysis

Finally, the next table summarizes the key numerical results, providing a direct comparison between the expected and executed trajectory metrics.

Metric	**Value**
Expected path length	23.5887 m
Executed path length	23.1044 m
Expected travel time	205.1284 s
Executed travel time	199.9332 s

Table 6.6: Expected versus executed trajectory metrics for Path 6

Chapter 7

Discussion and conclusions

The proposed approach has been successfully implemented, achieving all the intended objectives.
The environment was accurately mapped, and the path planning algorithm demonstrated its effectiveness even for images larger than 300×300 pixels.
Additionally, the control system proved to be efficient, as the expected values closely matched the actual results obtained from the simulations.

Upon closer analysis of a generic path executed by the TurtleBot3, an expected notable behavior emerges due to the use of the Pure Pursuit controller.
Specifically, the robot tends to slightly cut corners because of its tendency to anticipate the trajectory.
This behavior is a direct consequence of the look-ahead distance, which was set to 0.25 m.
As a result, particularly for longer trajectories, the actual path executed is often slightly shorter than the planned one.
This phenomenon could be mitigated by reducing the look-ahead distance, as a smaller value would make the robot follow the reference path more closely.
However, decreasing this parameter would also lead to excessive oscillations and a more unstable motion, making the trajectory less smooth and potentially affecting overall path-tracking performance.
An example of this behavior can be observed in the following zoomed-in view of Path 1.

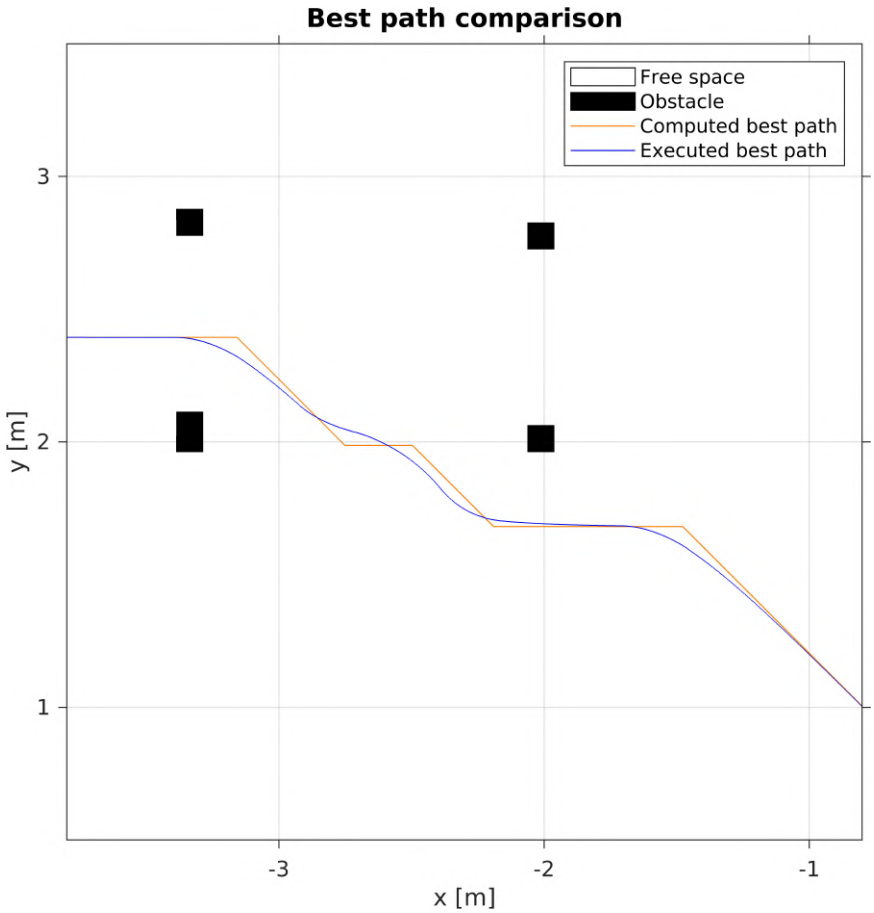

Figure 7.1: Effect of look-ahead distance on path execution

On the other hand, the executed path length may be underestimated at the beginning of some trajectories.
This occurs when the robot starts with an initial orientation different from the planned travel direction, causing it to follow a longer path to realign with the reference trajectory.
This phenomenon is evident in the starting phase of Path 3, as shown in the next figure.

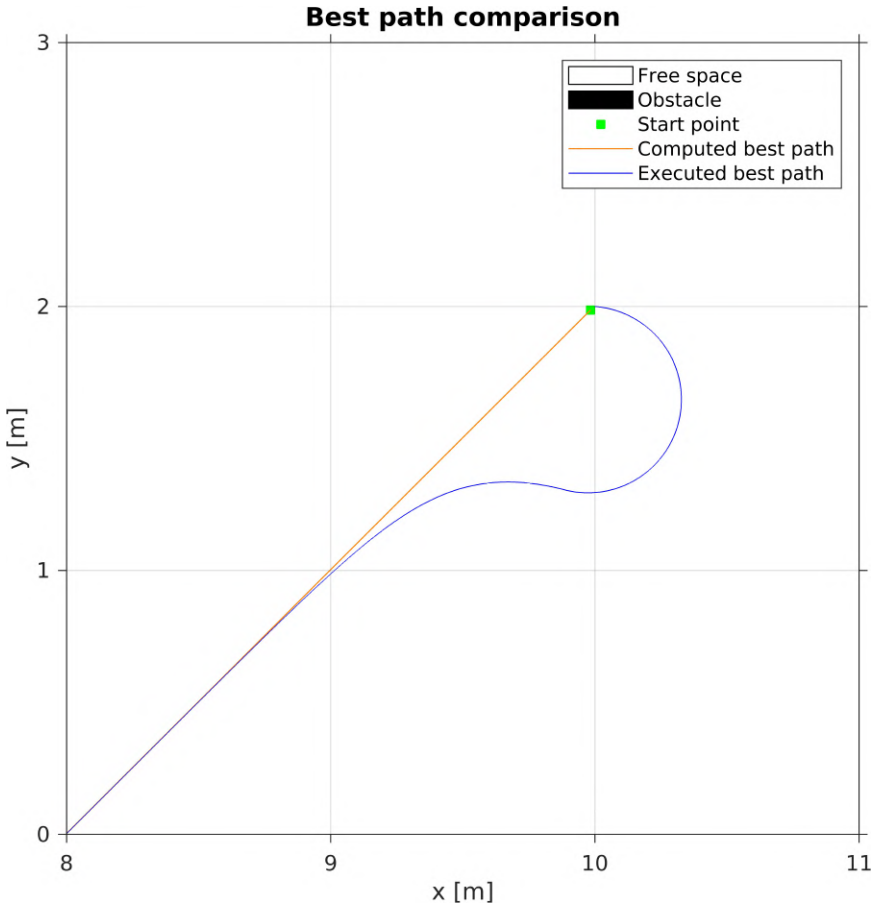

Figure 7.2: Initial realignment effect on path execution

Naturally, these deviations in path length are also reflected in the total time required to complete each trajectory.
Despite these minor discrepancies, the overall system behavior remains highly stable and reliable, demonstrating the robustness of the approach.

To conclude, while the proposed approach demonstrates effectiveness in controlled scenarios where a pre-mapped environment is available, a significant limitation arises from its dependence on prior knowledge of the environment for trajectory planning.
In real-world applications, obtaining a complete and accurate map before performing any navigation is not always feasible.
Environments are often dynamic, with obstacles and conditions that can change unpredictably, making it challenging to rely solely on pre-planned paths.
Although prior mapping can help improve performance in stable, known environments, it introduces added complexity and may not always be practical, particularly when dealing with unknown or rapidly changing surroundings.
Thus, real-time path planning that does not rely on prior map data was not implemented in this project, but it would be highly beneficial.
This capability would allow the system to adapt and respond to changes in the environment as they occur, without depending on a pre-existing map.
Implementing such an approach would improve the system's flexibility, enabling it to perform more effectively in dynamic, unpredictable, and complex environments. .